Spam Email Detector

This is a small machine learning project I built to detect whether a message is spam or not. I made this while learning about text classification and natural language processing (NLP) basics, and wanted a practical project to apply what I was learning instead of just doing theory.

Project Overview

The program takes a text message (like an email or SMS) and predicts whether it is "spam" or "ham" (a normal, legitimate message). It uses a Naive Bayes classifier trained on a small dataset I created myself, since I couldn't find a dataset I was comfortable redistributing in this repo.

This is not meant to be a production-ready spam filter, it's a learning project to understand how basic text classification works.

Features

- Loads and processes a labeled dataset of spam/ham messages

- Converts text into numerical features using TF-IDF
- Trains a Naive Bayes model to classify messages
- Prints accuracy, precision, recall, and F1 score after training
- Lets you test the model on your own custom message from the command line
- Saves the trained model so it doesn't need to be retrained every time

Technologies Used

- Python 3
- pandas (for loading/handling the dataset)
- scikit-learn (TF-IDF vectorizer, Naive Bayes model, evaluation metrics)
- joblib (for saving/loading the trained model)

Installation Steps

1. Clone this repository

```
git clone <your-repo-link>
cd spam_email_detector
```

2. (Optional but recommended) create a virtual environment

```
python -m venv venv
venv\Scripts\activate      # on Windows
source venv/bin/activate   # on Mac/Linux
```

3. Install the required packages

```
pip install pandas scikit-learn joblib
```

How to Run the Project

1. Generate the dataset (already included in the repo, but you can regenerate it):

```
cd dataset
python generate_dataset.py
cd ..
```

2. Train the model and see the evaluation results:

```
python spam_detector.py
```

3. Test the model on your own message:

```
python spam_detector.py --predict
"Congratulations, you have won a free
prize!"
```

Example Usage

```
python spam_detector.py --predict "Hey, are we still meeting later today?"
```

Output:

```
Message: Hey, are we still meeting later today?
Prediction: ham
```

```
python spam_detector.py --predict "URGENT: claim your free reward now"
```

Output:

```
Message: URGENT: claim your free reward now
Prediction: spam
```

What I Learned

This was my first time building an end-to-end ML project on my own, from data to a working model. A few things I learned along the way:

- How TF-IDF actually works, and why it's used instead of just counting words directly. Before this project I didn't fully get why raw word counts weren't enough.

- Why splitting data into train/test sets matters, I originally trained and tested on the same data and the accuracy looked way too good, which didn't make sense until I understood what was happening.
- How Naive Bayes handles text classification and why it's commonly used as a baseline model for this kind of task.
- Basic error handling and writing a script that can be run either directly or with command-line arguments.

One honest note: my model got every message right on the test set, but I don't think that means much here. My dataset is something I generated myself using templates, so the spam and ham messages are probably too easy to tell apart compared to real-world messages. I expect a real dataset would give lower and more realistic numbers, which is something I want to try next.

Future Improvements

- Use a real, publicly available dataset (like the SMS Spam Collection dataset) instead of my self-made one, to get a more honest measure of performance
- Try other models like Logistic Regression or SVM and compare results

- Add more features besides just the text, like message length or number of links
- Build a simple interface (maybe with Flask or Streamlit) instead of only running it from the command line
- Clean up the dataset generation script, right now it's fairly repetitive and could be organized better